Guía de ejercicios - Modelado computacional del lenguaje (v4)

Inteligencia Artificial y Neurociencias

Segundo semestre de 2026

Lenguajes formales

Ejercicio 0. Preliminares.

(a) Dada la cadena $x = abb$, calcular: x^0, x^1, x^2.

(b) Dado el lenguaje $L = \{ho, la\}$, calcular: L^0, L^1, L^2, L^*, L^+, $|L|$, $|L^0|$.

(c) Dado el lenguaje $L = \{ho, la\}$, decidir si las siguientes afirmaciones son verdaderas o falsas:
$\varepsilon \in L, \quad \varepsilon \in L^0, \quad \varepsilon \in L^1, \quad \varepsilon \in L^*, \quad \varepsilon \in L^+, \quad L^0 = \{\varepsilon\}$.

(d) Dados dos lenguajes $V = \{aa, b\}$ y $W = \{c, dd\}$, calcular:
$V \cup W, \quad V \cup W^*, \quad (V \cup W)^*, \quad VW, \quad V^*W, \quad (VW)^*, \quad (V \cup VW)^*$.

(e) Dar ejemplos de cadenas que pertenezcan a los siguientes lenguajes:
$\{a^k b^k \mid k \geq 0\}, \quad \{a^k b^{2k} \mid k \geq 1\}, \quad \{a^k b^j \mid 0 < k < j\}, \quad \{(ab)^k \mid k \geq 1\}, \quad \{(ab)^k \mid 0 \leq k \leq 3\}$.

Ejercicio 1. Escribir expresiones regulares para los siguientes lenguajes sobre $\Sigma = \{0, 1\}$:

(a) Cadenas de longitud par.

(b) Cadenas con cantidad par de ceros.

~~(c) Cadenas con cantidad par de ceros y cantidad impar de unos.~~

(d) Cadenas que comiencen con 00.

(e) Cadenas que terminen con 00.

(f) Cadenas que comiencen y/o terminen con 00.

(g) Cadenas que no contengan la subcadena 00.

(h) Cadenas que contengan la subcadena 00 al menos una vez.

Ejercicio 2. (Opcional) Considerar las siguientes igualdades para expresiones regulares r, s, t. Determinar cuáles son verdaderas (y demostrarlo) y cuáles son falsas (y encontrar un contraejemplo).

(a) $r + s = s + r$

(b) $r(s + t) = rs + rt$

(c) $(r + s)^* = r^* + s^*$

(d) $(r^*)^* = r^*$

(e) $(r^* + r) = r^*$

(f) $r + \varepsilon = r$

(g) $(\varepsilon + r)^* = r^*$

(h) $r + (st) = (r + s)(r + t)$

Ejercicio 3. Demostrar que toda gramática con producciones de la forma $A \to wB$ o $A \to w$ (donde $A, B \in V$, $w \in T^*$) puede reescribirse como una gramática regular, con producciones de la forma $A \to aB$, $A \to a$ o $A \to \varepsilon$ (donde $a \in T$).

Ejercicio 4. Escribir expresiones regulares equivalentes a las siguientes gramáticas regulares:

(a) $S \to 0S \mid 1S \mid 0 \mid 1$

(b) $S \to 0A \mid 1S \mid 0$
$A \to 0A \mid 0$

(c) $S \to 1A \mid 0S$
$A \to 1B \mid 0S \mid 1$
$B \to 0B \mid 1B \mid 0 \mid 1$

Ejercicio 5. Escribir gramáticas regulares para los lenguajes del ejercicio 1. Usar producciones de la forma $A \to wB$ o $A \to w$ (donde $A, B \in V$, $w \in T^*$), aprovechando el resultado del ejercicio 3.

Ejercicio 6. Demostrar que $L = \{w \mid w \in \{0,1\}^* \wedge w = w^R\}$ no es regular.

Ejercicio 7. En Python, el módulo `re` permite usar expresiones regulares. Para empezar a usarlo, alcanza con conocer tres funciones:

- `re.fullmatch(pattern, s)`, que determina si *toda* la cadena `s` pertenece al lenguaje de `pattern`;
- `re.search(pattern, s)`, que busca `pattern` en *alguna parte* de `s`; y
- `re.findall(pattern, s)`, que devuelve la lista de todas las coincidencias.

Las funciones `re.fullmatch` y `re.search` devuelven un objeto `Match` si hay coincidencia y `None` si no la hay, mientras que `re.findall` devuelve una lista de cadenas (vacía si no hay coincidencias).

Las expresiones regulares en Python tienen una sintaxis particular: la unión se escribe `|`, se usan `*`, `+` y `?` como operadores postfijos, `.` para cualquier símbolo, `[012]` para un símbolo entre varios, y `^` y `$` para el inicio y el fin de la cadena, respectivamente. Además, conviene poner `r` antes de cada string literal, para tratarlos como cadenas crudas y evitar las secuencias de escape.

(a) Sin ejecutar nada, predecir cuáles de las siguientes llamadas devuelven una coincidencia y cuáles `None`. Luego comprobarlo en el intérprete.

```
1 re.fullmatch(r(00|1)*, 1001)
2 re.fullmatch(r(00|1)*, 1010)
3 re.search(r00, 1010011)
4 re.fullmatch(r0*1?0*, 0010)
5 re.fullmatch(r[01]*00, 1100)
```

(b) Traducir a la sintaxis de Python las expresiones regulares del ejercicio 1 y revisarlas en Python.

(c) Dado el texto `t = Hay 12 gatos, 7 perros y 105 peces.`, usar `re.findall` para obtener:

- la lista de todos los números enteros (`[12, 7, 105]`);
- la lista de todas las palabras que terminan en `s`.

Ejercicio 8. Para cada uno de los siguientes lenguajes, dar una gramática libre de contexto que lo genere, elegir una cadena de longitud ≥ 4, y armar una derivación y un árbol de derivación según la gramática elegida.

(a) $\{a^n b^n \mid n \geq 0\}$

(b) $\{a^n b^m \mid m > n \geq 0\}$

(c) $\{a^n b^m \mid m \neq n \wedge m \geq 0 \wedge n \geq 0\}$

(d) $\{w \# w^R \mid w \in \{a,b\}^*\}$, donde w^R indica la reversa de w.

(e) $\{w \mid w \in \{a,b\}^* \wedge w = w^R\}$

(f) $\{w \mid w \in \{a,b\}^* \wedge w \neq w^R\}$

(g) $\{w \mid w \in \{a,b\}^* \wedge |w|_a = |w|_b\}$, donde $|w|_s$ es la cantidad de apariciones del símbolo s en la cadena w.

(h) $\{w \mid w \in \{a,b\}^* \wedge |w|_a > |w|_b\}$

(i) Cadenas de paréntesis balanceados.

Ejercicio 9. (Opcional) Sea L un lenguaje libre de contexto. Demostrar que:

(a) L^2 es un lenguaje libre de contexto.

(b) $\forall n \in \mathbb{N}, L^n$ es un lenguaje libre de contexto.

(c) L^* es un lenguaje libre de contexto.

(d) L^R es un lenguaje libre de contexto.

Ejercicio 10. Escribir una gramática libre de contexto que genere el lenguaje de las fórmulas bien formadas de la lógica proposicional, usando las constantes V y F, las variables p, q, r, s, los conectivos $\neg, \wedge, \vee$ y paréntesis.

Ejercicio 11. Una lista en Python se puede representar como una secuencia de elementos encerrados entre corchetes y separados por comas. Los elementos de una lista pueden ser a su vez listas. Dar una gramática libre de contexto que genere todas las listas posibles. Por simplicidad, considerar solamente el conjunto de símbolos terminales {`[`, `]`, `,`, `id`}, donde `id` son nombres válidos de variables. Es decir, no tener en cuenta literales ni expresiones compuestas.

Ejercicio 12. En Python, el módulo `nltk` permite definir y usar gramáticas libres de contexto. En `nltk.CFG`, los terminales van entre comillas simples, los no terminales sin comillas, y los símbolos de un cuerpo se separan con espacios. El método `parse` recibe la cadena como una lista de símbolos y devuelve los árboles de derivación posibles, o ninguno si la cadena no pertenece al lenguaje. Ejemplo:

```
import nltk
G = nltk.CFG.fromstring("""
S -> 'a' S 'b' | 'a' 'b'
""")
parser = nltk.ChartParser(G)
for arbol in parser.parse(list('aaabbb')):
    arbol.pretty_print()
```

(a) Antes de ejecutar, dibujar a mano el árbol de derivación de `aaabbb` según la gramática. Luego ejecutar el programa y comparar. Probar con las cadenas `aabbb` y `aacbb`.

(b) Considerar la gramática de expresiones aritméticas `E -> E + E | E * E | ( E ) | x`

- Parsear `x+x*x` y mostrar todos sus árboles de derivación. ¿Cuál de ellos corresponde a $(x + x) * x$ y cuál a $x + (x * x)$?
- Reescribir la gramática para que sea no ambigua y respete la precedencia de los operadores ($*$ se evalúa antes que $+$), usando tres no terminales `E`, `T` y `F`. Verificar que `x+x*x` tiene ahora un único árbol, y que su forma refleja la precedencia.

(c) Escribir en NLTK la gramática de fórmulas de lógica proposicional (ejercicio 10) y parsear tres cadenas válidas y tres inválidas. Para las inválidas, explicar por qué no hay árbol de derivación.

(d) (Opcional) Considerar la gramática $S \to S\,S \mid a$. Escribirla en NLTK y contar cuántos árboles de derivación tiene cada una de las cadenas a^k, para $k = 1, \ldots, 7$, usando `len(list(parser.parse(list('a' * k))))`. ¿Qué secuencia de números se obtiene? (Buscar números de Catalan.)

Atención: La evaluación de los temas de PLN y LLMs en el examen será mediante **preguntas conceptuales**, sobre los temas vistos en clase y desarrollados en la bibliografía indicada. En esta guía incluimos algunos ejercicios (*hands-on* y más o menos abiertos) que pueden ayudar a entender los temas, pero no necesariamente reflejan cómo será la evaluación en el examen.

Procesamiento del Lenguaje Natural

Ejercicio 13. Stemming vs. lematización: Comparar el stemmer Snowball (NLTK) y el lematizador de spaCy sobre un texto en español. Instalar los módulos `nltk` y `spacy` en Python y correr el siguiente código:

```
import spacy
from nltk.stem.snowball import SnowballStemmer

nlp = spacy.load("es_core_news_sm")
stemmer = SnowballStemmer("spanish")

texto = "Ayer fuimos al mercado y compramos las mejores naranjas."
for token in nlp(texto):
    print(token.text, stemmer.stem(token.text), token.lemma_)
```

Comparar los resultados obtenidos y correr otros ejemplos, para entender la diferencia entre stems y lemas.

Ejercicio 14. Analizar una oración con spaCy: clases de palabras (POS), entidades nombradas (NER) y dependencias sintácticas.

```
import spacy
nlp = spacy.load("es_core_news_sm")

doc = nlp("La Dra. Juana Pérez visitó el Banco Nación en Córdoba el 3 de marzo.")

for token in doc:
    print(token.text, token.pos_, token.dep_, token.head.text)

for ent in doc.ents:
    print(ent.text, ent.label_)
```

En la salida del programa:

1. Identificar el POS tag de cada palabra.
2. Ver qué entidades detecta y con qué etiquetas. ¿Reconoce la fecha? Averiguar por qué.
3. Construir el *árbol de dependencias* de la oración: una forma de representar la sintaxis de un texto, alternativa al arbol de derivación visto en clase. Para construirlo, buscar la palabra raíz indicada con `dep_='ROOT'`, y luego colgar recursivamente sus palabras hijas. Cada palabra indica en `token.head` cuál es su madre. Pensar cómo podría ser un árbol de derivación para esta oración (no conocemos la gramática, pero podemos imaginar como sería) y buscar cómo se relacionan los dos árboles.
4. Generar el árbol de dependencias en un notebook con `spacy.displacy.render(doc, style="dep")`. Comparar con el árbol construido a mano.
5. Repetir con textos potencialmente ambiguos (por ejemplo, *vi a una mujer con un telescopio*).

Ejercicio 15. El siguiente código entrena un modelo de lenguaje de bigramas con NLTK sobre 1000 artículos de Wikipedia en español, genera un texto a partir de *"la ciudad de"* y calcula la perplejidad del texto *"el río más largo del mundo"*. Requiere los módulos `nltk` y `datasets` de Python.

```
1  import nltk
2  from nltk.lm.preprocessing import padded_everygram_pipeline
3  from nltk.util import ngrams
4  from datasets import load_dataset
5  import os
6
7  # Descarga una única vez 1000 textos de Wikipedia y los guarda en un archivo.
8  archivo = "wiki_es.txt"
9  if not os.path.exists(archivo):
10     ds = load_dataset("wikimedia/wikipedia", "20231101.es",
11                       split="train", streaming=True)
12     with open(archivo, "w", encoding="utf-8") as f:
13         for a in ds.take(1000):
14             f.write(a["text"] + "\n")
15
16 # Lee los textos de Wikipedia.
17 with open(archivo, encoding="utf-8") as f:
18     texto:str = f.read()
19
20 # Separa los textos en oraciones.
21 nltk.download("punkt")
22 oraciones:list[str] = nltk.sent_tokenize(texto, language="spanish")
23
24 # Convierte cada oración a una lista de strings.
25 tokens_por_oracion:list[list[str]] = [
26     [w.lower() for w in nltk.word_tokenize(o, language="spanish")]
27     for o in oraciones
28 ]
29
30 # Construye un modelo de n-gramas.
31 n = 2
32 train, vocab = padded_everygram_pipeline(n, tokens_por_oracion)
33 model = nltk.lm.MLE(order=n)
34 model.fit(train, vocab)
35
36 # Usa el modelo de n-gramas construido.
37 print(model.generate(20, text_seed=["la", "ciudad", "de"]))
38 print(model.score("mundo", ["del"]))   # P(mundo | del)
39 toks = "el río más largo del mundo".split()
40 print(model.perplexity(ngrams(toks, n)))
```

1. Modificar el código para entrenar modelos con $n = 2$ y $n = 3$ y generar texto a partir de *la ciudad de*. ¿Cómo cambia la fluidez del texto?

2. Buscar la definición de *perplejidad* de un modelo de lenguaje. Calcular la perplejidad de *"el río más largo del mundo"* y de *"el mundo más largo del río"* con cada modelo. ¿Cuál prefiere cada uno?

3. Repetir con otros pares de oraciones inventadas, como por ejemplo:

   - *"el presidente tuvo que renunciar"* vs. *"el presidente tubo que renunciar"*
   - *"el orden de las palabras es importante"* vs. *"de el es importante las orden palabras"*

   (Si alguna perplejidad devuelve infinito, leer el inciso siguiente.)

4. Averiguar qué es el *smoothing* (suavizado) y por qué es necesario. Reemplazar la línea 33 por:

   ```
   model = nltk.lm.WittenBellInterpolated(order=n)
   ```

   y comparar los resultados.

Ejercicio 16. Dados los siguientes documentos y vocabulario, construir la matriz de frecuencias y comparar documentos y palabras mediante similitud coseno.

Documentos:

D1: El banco aumentó el interés del crédito.
D2: Pedí un crédito al banco para tener dinero.
D3: El dinero ahorrado genera interés en el banco.
D4: Me senté en un banco de la plaza, a la sombra de un árbol.
D5: En la plaza, el perro duerme a la sombra.
D6: El perro descansa bajo un árbol de la plaza.

Vocabulario: banco, dinero, crédito, interés, plaza, árbol, sombra, perro.

El siguiente código construye la matriz de frecuencias con scikit-learn y calcula las similitudes coseno entre todos los pares de documentos y palabras del vocabulario. Requiere los módulos `pandas` y `sklearn` de Python.

```python
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics.pairwise import cosine_similarity

docs = ["El banco aumentó el interés del crédito.", ...]
vocab = ["banco", "dinero", "crédito", "interés",
         "plaza", "árbol", "sombra", "perro"]

# Crea una matriz de frecuencias de 6x8: una fila por documento y
# una columna por palabra del vocabulario.
X = CountVectorizer(vocabulary=vocab).fit_transform(docs).toarray()
print(pd.DataFrame(X, columns=vocab))

print(cosine_similarity(X).round(2))      # documentos
print(cosine_similarity(X.T).round(2))    # palabras
```

1. Construir a mano la matriz documento-término y verificarla con el código.
2. Calcular a mano la similitud coseno entre *dinero* y *crédito*, y entre *dinero* y *perro*.
3. ¿Qué par de documentos es más similar? ¿Y menos similar? ¿Coincide con tu intuición?
4. ¿Qué palabras son las más similares a *banco* y a *perro*?

Grandes Modelos de Lenguaje (LLMs)

Ejercicio 17. Tokenización con *byte-pair encoding* (BPE). Considerar el siguiente corpus, donde cada palabra aparece con la frecuencia indicada:

Palabra	Frecuencia
casa	3
casas	2
caso	1

Inicialmente, cada carácter es un token independiente. Suponer que el vocabulario inicial contiene todas las letras minúsculas del alfabeto. BPE agrega iterativamente la fusión del par de tokens adyacentes más frecuente en el corpus.

(a) Realizar **tres iteraciones de BPE**. Indicar qué par se fusiona en cada paso.

(b) Mostrar cómo queda tokenizado el corpus después de los tres merges.

(c) ¿Cómo se tokenizaría la palabra nueva *"casita"* utilizando los merges aprendidos?

(d) ¿Por qué BPE permite representar palabras que nunca aparecieron en el corpus de entrenamiento? ¿Qué ventajas tiene frente a utilizar palabras completas como tokens?

Ejercicio 18. Utilizar un tokenizador existente (por ejemplo, de Hugging Face o `tiktoken`) para analizar cómo distintos textos son convertidos a tokens. Comparar al menos los siguientes casos:

Categoría	**Ejemplos**
Inglés vs. español	"Are you coming tomorrow?" / "¿Venís mañana?"
Tildes y voseo	"tú vienes mañana" / "vos venís mañana"
Errores de escritura	"mañana" / "maniana"
Números	"1000000" / "1.000.000"
Emojis	"Estoy feliz" / "Estoy ☺"

(a) Mostrar los tokens generados y su cantidad en cada caso.

(b) Comparar la cantidad de tokens para frases parecidas en inglés y español. ¿Qué diferencias hay?

(c) ¿Cómo puede afectar la tokenización al costo computacional y económico de utilizar un LLM?

(d) ¿De qué manera podría introducir sesgos o diferencias de rendimiento entre idiomas, dialectos y formas de escritura?

Ejercicio 19. Mecanismo de atención. Considerar la frase *"banco del río"*, representada por tres tokens. En una cabeza de atención de dimensión $d_k = 2$, se tienen las siguientes matrices:

$$Q = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad K = \begin{pmatrix} 0 & 1 \\ 0 & 1 \\ 2 & 0 \end{pmatrix}, \qquad V = \begin{pmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 1 \end{pmatrix}$$

La operación de atención es:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V$$

donde M representa una máscara aditiva y el softmax se aplica por filas.

Recordar que la matrix 3x3 de máscara bidireccional y causal son distintas y que $\exp(-\infty) = 0$.

(a) Calcular la matriz de puntajes $\frac{QK^\top}{\sqrt{d_k}}$ y los pesos de atención resultantes, aplicando softmax por fila.

(b) Calcular la salida de atención para el token *"banco"*, primero sin máscara y luego utilizando la máscara causal.

(c) ¿A qué token presta mayor atención *"banco"* sin máscara? ¿Cómo cambia su representación al aplicar la máscara causal? ¿Por qué es importante esta diferencia entre modelos bidireccionales y autoregresivos?

(d) Suponiendo que los componentes de q y k son independientes, de media cero y varianza uno, justificar por qué los puntajes se dividen por $\sqrt{d_k}$. ¿Qué sucede con la varianza de los productos escalares y con el softmax si no se realiza este escalado?

Ejercicio 20. La figura 7.24 del libro de Jurafsky & Martin (también disponible al final de la primera clase de LLMs), muestra una tabla con las constantes L, d, A, d_{ff} etc., de algunos modelos GPT, Qwen y Llama.

(a) Hacer las cuentas para llegar a la cantidad de parámetros reportada en cada caso.

(b) ¿Por qué el tamaño del contexto (N) no incide en el tamaño del modelo?

(c) ¿Cómo es la complejidad computacional de la inferencia respecto de N, suponiendo fija la arquitectura?

Ejercicio 21. Para cada una de las siguientes tareas, elegir entre una arquitectura **encoder-only**, **decoder-only** o **encoder-decoder**, y justificar la decisión.

Tarea	**Arquitectura**
Clasificar el sentimiento de una reseña	
Traducir un texto del español al inglés	
Autocompletar un mensaje	
Identificar entidades nombradas (NER)	
Resumir una noticia	

(a) Explicar si cada tarea requiere principalmente comprender una entrada, generar texto o ambas cosas.

(b) Indicar cómo se produciría la salida en cada caso.

(c) ¿Es posible resolver todas estas tareas usando un modelo decoder-only? Pensar ventajas y desventajas.

Ejercicio 22. Investigar qué tipo de aplicaciones concretas tiene hoy en día la técnica RAG.

Ejercicio 23. En julio de 2025, el chatbot Grok publicó en X contenido antisemita, violento y conspiranoico, lo cual fue denunciado por miles de usuarios. Con lo visto en la materia, proponé un explicación técnica posible para ese comportamiento.